

Build Instructions

Complete step-by-step guide to building the Space Shooter game for Windows.

Prerequisites

1. **Unity Hub** (Latest version)
 - Download: <https://unity.com/download>
 2. **Unity Editor** (2021.3 LTS or newer recommended)
 - Install via Unity Hub
 - Include “Windows Build Support” module
 3. **Visual Studio** or **Visual Studio Code**
 - For C# script editing
 - Install via Unity Hub or separately
-

Step 1: Create New Unity Project

1. Open Unity Hub
 2. Click “New Project”
 3. Select template: **2D (Built-In Render Pipeline)**
 4. Project name: `SpaceShooter`
 5. Location: Your preferred folder
 6. Click “Create Project”
-

Step 2: Import Scripts

Copy Script Files

1. In Unity, right-click in Project window
2. Create folders matching the structure:

```
Assets/  
├── Scripts/  
│   ├── Core/  
│   ├── Player/  
│   ├── Enemy/  
│   ├── Combat/  
│   ├── Systems/  
│   ├── UI/  
│   └── Audio/  
├── Prefabs/  
└── Scenes/
```

└─ Sprites/

└─ Audio/

3. Copy all `.cs` files from the provided project into their respective folders:
 - `Core/` → `GameManager.cs`, `ObjectPooler.cs`
 - `Player/` → `PlayerController.cs`, `PlayerShield.cs`
 - `Enemy/` → `EnemyBase.cs`, `BossEnemy.cs`, `EnemySpawner.cs`
 - `Combat/` → `Bullet.cs`, `HealthSystem.cs`, `DamageOnContact.cs`
 - `Systems/` → `WaveSpawner.cs`, `ScoreManager.cs`, `PowerUp.cs`, `PowerUpSpawner.cs`, `ParallaxBackground.cs`, `StarfieldGenerator.cs`
 - `UI/` → `UIManager.cs`, `MainMenu.cs`, `HealthBarUI.cs`
 - `Audio/` → `AudioManager.cs`, `SoundEffectPlayer.cs`
 4. Wait for Unity to compile scripts (check console for errors)
-

Step 3: Install TextMeshPro

1. Go to Window > Package Manager
 2. Search for "TextMeshPro"
 3. Click "Install" if not already installed
 4. When prompted, click "Import TMP Essentials"
-

Step 4: Configure Project Settings

Tags

1. Edit > Project Settings > Tags and Layers
2. Add tags:
 - Player
 - Enemy
 - PlayerBullet
 - EnemyBullet
 - PowerUp

Layers

1. In same window, add layers:
 - Layer 8: Player
 - Layer 9: Enemies
 - Layer 10: PlayerBullets
 - Layer 11: EnemyBullets
 - Layer 12: PowerUps

Sorting Layers

1. Add sorting layers (in order):
 - Background
 - Stars
 - Projectiles
 - Pickups

- Characters
- Effects

Physics 2D

1. Edit > Project Settings > Physics 2D
 2. Configure Layer Collision Matrix:
 - Uncheck all by default
 - Player ↔ Enemies: ✓
 - Player ↔ EnemyBullets: ✓
 - Player ↔ PowerUps: ✓
 - Enemies ↔ PlayerBullets: ✓
-

Step 5: Create Sprites (Placeholders)

Using Unity's Built-in Sprites

1. Right-click in Sprites folder
2. Create > 2D > Sprites > Choose shape

Or Import Your Own

1. Drag PNG files into Assets/Sprites folder
2. Select sprite, in Inspector:
 - Sprite Mode: Single
 - Pixels Per Unit: 100
 - Filter Mode: Point (for pixel art)
 - Apply

Required Sprites:

- player_ship
 - bullet_player
 - bullet_enemy
 - enemy_basic
 - enemy_zigzag
 - enemy_circular
 - enemy_charger
 - enemy_boss
 - powerup
 - background (optional)
-

Step 6: Create Prefabs

Player Prefab

1. Create empty GameObject, name "Player"
2. Add components:
 - Sprite Renderer (assign player sprite)

- Rigidbody2D (Gravity Scale: 0, Freeze Rotation Z)
 - Box Collider 2D (Is Trigger: true)
 - PlayerController script
 - HealthSystem script
 - AudioSource
3. Create child "FirePoint" at (0, 0.5, 0)
 4. Set tag: "Player", Layer: "Player"
 5. Drag to Prefabs folder

Player Bullet Prefab

1. Create GameObject "PlayerBullet"
2. Add:
 - Sprite Renderer (cyan color)
 - Rigidbody2D (Gravity: 0)
 - Circle Collider 2D (Is Trigger: true)
 - Bullet script (Is Player Bullet: true)
3. Set tag: "PlayerBullet", Layer: "PlayerBullets"
4. Drag to Prefabs folder

Enemy Bullet Prefab

1. Duplicate PlayerBullet, rename "EnemyBullet"
2. Change color to red
3. Bullet script: Is Player Bullet: false
4. Set tag: "EnemyBullet", Layer: "EnemyBullets"
5. Drag to Prefabs folder

Enemy Prefabs

For each enemy type (Basic, Zigzag, Circular, Charger):

1. Create GameObject
2. Add:
 - Sprite Renderer (different color each)
 - Rigidbody2D (Kinematic)
 - Collider 2D (Is Trigger: true)
 - EnemyBase script (set Enemy Type)
 - HealthSystem script
3. Set tag: "Enemy", Layer: "Enemies"
4. Drag to Prefabs folder

Boss Prefab

1. Same as enemy but use BossEnemy script
2. Make sprite larger
3. Higher health (500)

PowerUp Prefab

1. Create GameObject "PowerUp"
2. Add:
 - Sprite Renderer (white, color set by script)

- Circle Collider 2D (Is Trigger: true)
 - PowerUp script
3. Set tag: "PowerUp", Layer: "PowerUps"
 4. Drag to Prefabs folder
-

Step 7: Create Scenes

MainMenu Scene

Follow the SCENE_SETUP.md guide for MainMenu setup.

1. File > New Scene > Save as "MainMenu"
2. Set up camera, canvas, UI elements
3. Add MainMenu script to manager object

GameScene

Follow the SCENE_SETUP.md guide for GameScene setup.

1. File > New Scene > Save as "GameScene"
 2. Add all managers, spawners, player, UI
 3. Connect all references
-

Step 8: Configure Build Settings

1. File > Build Settings
2. **Add Scenes:**
 - Click "Add Open Scenes" or drag scenes:
 - Scenes/MainMenu (must be index 0)
 - Scenes/GameScene (index 1)
3. **Platform Settings:**
 - Select "PC, Mac & Linux Standalone"
 - Target Platform: Windows
 - Architecture: x86_64 (recommended)
4. **Player Settings (click button):**

Product Name: Space Shooter

Company Name: Your Name

Version: 1.0.0

Icon: (optional) drag your icon image

Resolution:

- Default Is Fullscreen: Unchecked (for testing)
- Default Screen Width: 1920

- Default Screen Height: 1080
- Or: Fullscreen Window

Other Settings:

- API Compatibility Level: .NET Standard 2.1
 - Scripting Backend: Mono
-

Step 9: Build the Game

Development Build (for testing)

1. File > Build Settings
2. Check "Development Build"
3. Check "Script Debugging" (optional)
4. Click "Build"
5. Create new folder: Builds/Windows
6. Name the executable: SpaceShooter.exe
7. Click "Save"

Release Build

1. Uncheck "Development Build"
 2. Uncheck "Script Debugging"
 3. Click "Build"
 4. Choose output folder
 5. Wait for build to complete
-

Step 10: Test the Build

1. Navigate to build folder
 2. Run SpaceShooter.exe
 3. Test all features:
 - [] Main menu loads
 - [] Play button starts game
 - [] Player moves with WASD/Arrow keys
 - [] Player shoots with Space
 - [] Enemies spawn in waves
 - [] Collision detection works
 - [] Score updates
 - [] Power-ups work
 - [] Pause menu (ESC)
 - [] Game over screen
 - [] Quit button exits
-

Build Output Structure

```
Builds/
├── Windows/
│   ├── SpaceShooter.exe           ← Run this!
│   ├── SpaceShooter_Data/        ← Game data folder
│   │   ├── Managed/
│   │   ├── Resources/
│   │   ├── StreamingAssets/
│   │   └── ...
│   ├── MonoBleedingEdge/
│   └── UnityPlayer.dll
```

Troubleshooting

Build Errors

“Script has compilation errors”

- Open Console (Window > General > Console)
- Fix any red error messages in scripts
- Save all scripts and rebuild

“Scene not found”

- Ensure scenes are added to Build Settings
- Check scene names match (case-sensitive)

Missing references

- Check Inspector for missing (None) fields
- Reassign prefabs and references

Runtime Errors

Player doesn’t move

- Check Input settings (Edit > Project Settings > Input Manager)
- Ensure PlayerController is on Player object

Bullets don’t damage enemies

- Check tags are set correctly
- Check colliders are triggers
- Check layer collision matrix

UI doesn’t show

- Ensure Canvas is in scene
- Check UIManager references
- Verify panels are active/inactive correctly

Distribution

Zip for Distribution

1. Copy entire build folder

2. Compress to ZIP
3. Include README with:
 - System requirements
 - Controls
 - How to play

System Requirements

- OS: Windows 7/8/10/11 (64-bit)
 - Processor: 1.5 GHz or faster
 - Memory: 2 GB RAM
 - Graphics: DirectX 11 compatible
 - Storage: 100 MB
-

Optional: Code Signing

For professional distribution:

1. Obtain code signing certificate
 2. Use signtool.exe to sign the executable
 3. This removes “Unknown publisher” warnings
-

Quick Build Checklist

- ☐ All scripts compile without errors
- ☐ All prefabs created with correct components
- ☐ All prefab references assigned
- ☐ Tags and layers configured
- ☐ Both scenes created and added to Build Settings
- ☐ MainMenu is scene index 0
- ☐ Player settings configured
- ☐ Test in editor first
- ☐ Build created successfully
- ☐ Built game tested and working